A Random Walk Through the Web

Network Analysis of the Google Web Graph

Richard Zhang, Max Peng, Toby Liu, Frank Wang

PSTAT 194CS — Social Network Analysis
University of California, Santa Barbara

March 2026

Introduction

The internet has become one of the most transformative and essential technologies in the early AI era. Serving as a universal learning tool, research platform, and the foundational infrastructure where AI continues to advance. With the rapid rise of AI, the internet has never been more critical, powering data pipelines, knowledge bases, and interconnected systems that modern machine learning systems depend on. Yet to truly understand the internet as it is today, it is valuable to examine it in its early development. Particularly in 2002, during a turbulent times for the internet as it marked the aftermath of the dotcom bubble. Which led to many internet-based companies to rise and fall almost instantly. Fundamentally reshaping how the internet is today.

Understanding how web pages were interconnected during this era not only reveals the structural properties of the early internet but also sheds light on how information flowed, how communities of related pages formed, and how search engines like Google first learned to navigate and rank content in the wake of one of the most dramatic technological disruptions in modern history.

Dataset Description

We will be conducting a social network analysis to the web-Google dataset. Which is a directed web graph released by Google in 2002 as part of the Google Programming Contest. Made publicly available through the Stanford Network Analysis Project (SNAP). Where the dataset comprises of 875713 webpages (vertices) and 5105039 directed hyperlinks (edges) capturing a rich and large-scale portait of the internet as it existed in 2002. In which, the vertices represent individual web pages that were collected from the internet in 2002. With each node corresponding to a unique webpage labeled by a numeric id, and edges representing directed hyperlinks between webpages.

Table 1: Basic Network Elements of the web-Google Network

Property	Value
Nodes	875713
Edges	5105039
Directionality	Yes
Weighted	No
Self-loops	No
Density	6.7e-06
Mean Degree	11.66
Max Degree	6353

Objectives:

The objectives of this analysis are to: * Generate representative subgraphs of the Google Web graph using random walk sampling to deal with sparsity limitations of induced subgraphs * Look into sampling bias by comparing degree distributions between the sampled subgraph and full network using the Kolmpgorov-Smirnov test * Visualize the network structure of our subgraphs with the Fruchterman-Reingold, Kamada-Kawai, and Multidimensional Scaling (MDS) layout to reveal structural and connectivity patterns * Compute and interpret network-level metrics (i.e. density, reciprocity, dyad census, transitivity, diameter, and mean distance) to capture the overall structure of the web graph * Compute and interpret node-level metrics (i.e. in-degree, out-degree, betweenness centrality, closness centrality, and local transitivity) to identify influential web pages within the network * Detect and compare communities using Fast-Greedy, Louvain, Walktrap algorithms and evaluate their results using molecularity scores and community sizes * Examine the adjacency matrix of each subgraphs in their original and reordered form to confirm the presence of community strucutre and identify structural patterns.

Methodology

Creating Subgraphs

We will first generate four induced subgraphs by "purely" randomly sample 200 nodes from the full network just using the `induced_subgraph()`.

```
## IGRAPH 77c5521 DN-- 200 1 --
## + attr: name (v/c)
## + edge from 77c5521 (vertex names):
## [1] 860469->343525
```

```
## IGRAPH 34680f7 DN-- 200 0 --
## + attr: name (v/c)
## + edges from 34680f7 (vertex names):
```

```
## IGRAPH 11246a1 DN-- 200 0 --
## + attr: name (v/c)
## + edges from 11246a1 (vertex names):
```

```
## IGRAPH 05659b2 DN-- 200 1 --
## + attr: name (v/c)
## + edge from 05659b2 (vertex names):
## [1] 203345->306718
```

However, we can notice that from the results above, our subgraphs virtually do not have any edges. This could be due to the fact that the web-Google graph is extremely sparse (density $\approx 6.7 \times 10^{-6}$). This would make it difficult to get any meaningful analysis.

To resolve this issue, we will utilize the concept of random walk to sample. Where we would use igraph's built in `random_walk()` function. Beginning with a randomly chosen seed node, a random walk proceeds for 500 steps by following the network's edges and recording the nodes encountered. The first 200 unique nodes visited during this process are then used to construct each induced subgraph. This method helps ensure that the resulting subgraphs are well connected and contain many edges, maintaining the local hyperlink structure of the web.

Code Implementation

Table 2: Summary of Random Walk Subgraphs

Subgraph	Nodes	Edges
Subgraph 1	200	957
Subgraph 2	200	970
Subgraph 3	200	842
Subgraph 4	200	1608

Based on the results above, we can see that our random walk sampling helped with providing us with edges for our all four of our subgraphs.

Checking on Sampling Bias

Though our subgraphs have edges using random walk sampling, there are still sampling biases that we need to consider.

Table 3: Sampling Bias Assessment of Random Walk Subgraphs

Subgraph	Nodes	Edges	Mean Degree	Median Degree	KS P-Value
Subgraph 1	200	957	9.57	7	0.0000337
Subgraph 2	200	970	9.70	5	0.0062651
Subgraph 3	200	842	8.42	6	0.0000047
Subgraph 4	200	1608	16.08	12	0.0000000

By comparing the degree distributions of each subgraph against the full network using summary statistics and the Kolmogorov-Smirnov (KS) test. Looking into our mean degree across subgraphs, we can notice that they range from 8.42 to 16.08. Where our median degree range from 5 to 12. Which suggests that some walks landed in denser regions of the network than others. Particularly subgraph 4. As it is considerably denser than the other subgraphs. Indicating that the random walk from subgraph 4 traversed a more tightly connected neighborhood of the web graph. This is important as we will be utilizing subgraph 4 to observe network metrics later in our report.

When observing our KS test p-values, we can see that all four subgraphs result in extremely small p-values. Indicating that a statistically significant difference between the subgraphs and the full network degree distributions. This is not bad as this is expected given the massive size difference between 200 nodes and the full roughly 876000 nodes. With this scale the KS test is extremely sensitive. Flagging even the smallest differences as significant. Thus our random walk sampled subgraphs are still suitable for structural analysis. As it still preserves the local connectivity patterns of the web graph rather than attempting to replicate its global degree distribution.

Tools and Packages Used

Table 4: R Packages Used

Package	Purpose
`igraph`	Graph construction, manipulation, metrics and visualization
`R.utils`	Decompressing the raw .gz dataset file
`knitr`	Generating formatted tables and controlling RMarkdown output

Analysis

Visualization

Now to get a better understanding of our subgraphs that we have created, we will visualize them using three various layouts available in the `igraph` package. With those three layouts being **Fruchterman-Reingold (FR)**, **Kamada-Kawai (KK)**, and **Multidimensional Scaling (MDS)**. In which these layouts use different approaches to position nodes in a two-dimensional space. Giving a different perspective on our subgraph network structure. With the FR layout being useful in revealing clusters and overall community structure of a network. The KK layout positions nodes so that their geometric distance on the plot reflecting their shorted path length. With the nodes close in the network to appear visually closer. It is also better at preserving the true relational distances between nodes than teh FR layout. Lastly, the MDS layout, will take the matrix of the shortest distances between all pair nodes and project it into a 2D space, to minimize distortion. With it emphasizing the global distance structure of the network.

Subgraph 1

Across all three layouts, we can notice that subgraph 1 reveals a fragmented structure consisting of several distinct dense clusters connected by sparse bridge nodes rather than a single cohesive core. With the FR

layout, we can see that it displays a central high populated region where many node connect, several branching chains extending outward from the core, and small satellite clusters at the end of the branches. In the KK layouts, we can see that the core cluster is very tight and circular with the branches appearing more spread and structured compared to the FR layout. Lastly in MDS layout, we can notice that the branches stretch our more linearly, the central cluster is less visually dense, and the network forms a Y like structure with the lines having relatively equal length. With these three layouts, it reveals that subgraph 1 contains a central core with several peripheral branches. Which tells us that the subgraph 1's network has localized clustering and hierarchical link structures. In the context of webpages and hyperlinks, this structure suggests that the web graph contains clusters of closely related pages with dense internal linking, while a limited number of bridging pages connect otherwise separate regions of the network.

Subgraph 2

Based on the visuals, we can see that it is relatively different different compared to subgraph 1. In the FR Layout, we can notice that it has the same U shape as the FR layout for subgraph 1. However, we can see that there are visibly two clusters. With these clusters being connected by a long chain of nodes. Suggesting a bridge like path linking between two communities. Which also indicate that the connectivity between the two groups is weak. With the KK layout, we also notice two clusters much clearer. With it also showing a connecting path between the two communities, and peripheral node branching off the main chain. Which indicate the same thing as the FR layout. Showing that we have two groups that are structurally separate from each other but are connected. In the MDS layout, we can see that like the visuals from subgraph 1, it has a Y shape. However, subgraph 2 is upside down. We can also notice that most of our nodes are on the bottom two lines of the upside down Y. When viewed as webpages and hyperlinks, this pattern indicates that two groups of pages are mostly internally connected, with only a few bridging pages linking the two sections of the network.

Subgraph 3

For the visuals for subgraph 3, we observe how different they look compared to the other two subgraphs above. For the FR layout, like subgraph 2, we can see that we have two clusters that are weakly connected to each other. However, we notice that the link between the two are more direct and the clustering is a little bit weaker compared to subgraph 2. Also it does not display a U shape. This also shows that there are only a few nodes that link between the two groups. In the KK layout, we can also see two communities that are connected to each other. However, we can see that the clustering of the two clusters are relatively weaker (nodes are more spread out). Lastly, for the MDS layout, we can see that it is also a Y shape like the other subgraphs we have seen above. However, we can notice some nodes that are not tightly within the Y shape. Which reveals that the network contains several long paths branching from a central region. Interpreting nodes as webpages and edges as hyperlinks, this structure suggests that the network contains two loosely connected groups of webpages, where most links occur within each group and only a few intermediary pages provide connections between them.

Subgraph 4

Lastly for subgraph 4, we can also clearly notice that we have two dense clusters connected bu a narrow bridging path. Where in the FR layout, can can see that we have two clusters. However, in those clusters there are subclusters in them. As we can see small clusters spread around in both the overall large clusters. As for the K layout we can also see that we have two groups. However, it shows the same results are the FR layout. In that there are several small subclusters that appear inside the larger group. Which also highlight how our community is structured. Showing that nodes within the cluster are more strongly connected to each other than to nodes in the other group. Now what is extremely interesting is the MDS layout. Where for the other subgraphs they for the most part formed a tight Y shape. However, for subgraph 4 we can notice how it does form a Y shape. But, the nodes are extremely spread out. This suggests that our network is organized more around paths and branching connections rather than tightly packed clusters. In relation to webpages and hyperlinks, this structure suggests that webpages are organized into clustered sections with internal

linking, while a small number of bridging pages create branching hyperlink paths that connect different parts of the network.

Network Metrics

Now, we will focus our analysis on the network metrics of subgraph 4 (as mentioned previously) in order to quantitatively examine the structural properties suggested from the visualizations we have examined. While the layouts provide useful qualitative insights into clustering, bridging node, and branching paths, we can gain further insights by measuring those characteristics. Such as density, reciprocity, transitivity, path lengths, clustering behavior, and importance of individual nodes within subgraph 4.

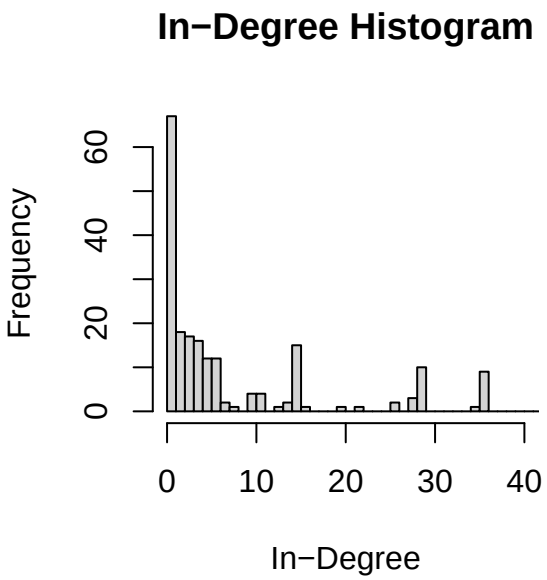
Metric	Value
Density	0.0404
Reciprocity	0.4279
Transitivity	0.6703
Mean Distance	2.9142
Diameter	8.0000
Dyad Census: Mutual	344.0000
Dyad Census: Asymmetric	920.0000
Dyad Census: Null	18636.0000
Closeness Centralization	0.1463
Betweenness NaN Proportion	0.0000
Closeness NaN Proportion	0.0550

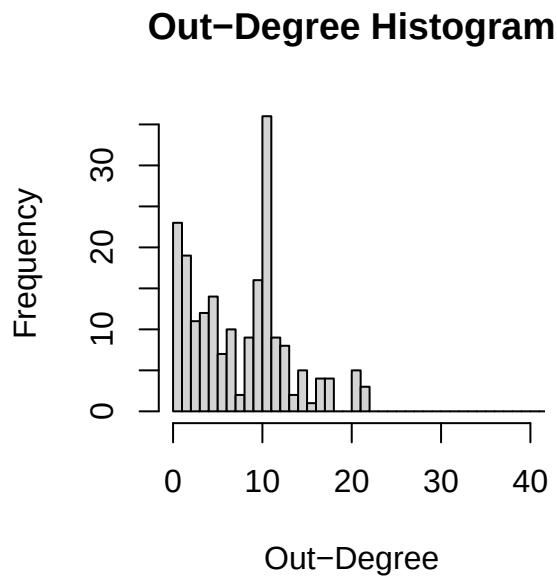
Density is the proportion of edges in a network compared to how many possible edges could exist. A density of 0.040402 indicates a sparse network with a low proportion of connected nodes. This is reasonable for a Google web graph, as only somewhat-related websites would have links connecting one to the other.

Reciprocity is the proportion of edges from node A to B that have a corresponding edge traveling from B to A. This quantity tends to be higher in friendship networks where connections are mutual, and lower in follower networks due to their one-sided nature. Our value of 0.4278607 suggests a moderately low percentage of websites with mutual hyperlinks. The dyad census takes all pairs of nodes and calculates the number of mutual connections, one-sided (asymmetric) connections, and null connections among all node pairs. We see that there are 344 mutually connected nodes and almost 20000 unconnected nodes. This echoes the low density found from earlier, and is perfectly reasonable since hyperlinks should be relevant to the website it was found on.

Diameter measures the length of all shortest paths between nodes in the network and returns the maximum value, while average distance returns the mean. Higher values indicate more indirect connections. A mean distance of 2.9142355 is quite low, and suggests that connections seldom occur through a large number of intermediary websites.

Global transitivity is the likelihood that two different neighbors of a node also share an edge. High transitivity indicates a network with tight-knit groups and more clustering. This is the case for our network, as a transitivity of 0.6703389 shows the close-knit nature of websites with hyperlinks conencting each other, even on a global scale.





Degree is a node-level measure of how many edges connected to it. Since we are working with a directed graph, the number of in-edges and out-edges are distinct. There is a clear right skew in both histograms, which is expected for a web graph since most websites will have a handful of connections but few will have a great deal.

Betweenness (per node) Betweenness is a centrality measure that finds the frequency of a node existing within the shortest path between two other nodes. Nodes with high betweenness centrality are often critical points in the network that control information flow. In our network some nodes have a betweenness measure of over 1400 while others have a betweenness of 0. Pages like Yahoo or trusted encyclopedia pages likely had these high centralities, since they would have been referred to frequently by other sites even if they were not directly related.

Closeness Centrality (per node) Closeness is a centrality measure that measures the distance from a node to all other nodes. Important nodes will have a higher closeness since they interact with the other nodes more closely. The closeness values typically fall between 0.15 and 0.25, meaning that websites take around 4-7 hyperlinks to jump to the next website. The theme of close-knittedness between websites persists here; website hyperlinks will typically be relevant to the original website and most connections will not stray too far away. The low centralization of <0.15 suggests that centrality is distributed fairly evenly across the network, with very few websites dominating traffic.

Local Transitivity (per node) Local transitivity is the smaller-scale measure of transitivity, and tells whether the data's clustering occurs frequently between small groups. Values frequently take values close to 1, which again shows that web graphs are tightly connected on a local level. Sites covering related topics will likely reference each other.

Community Detection

Because the community detection algorithms want an undirected graph and our current graph is directed, we need to convert it into an undirected graph first. Then, we can run all of the community detection algorithms on each subgraph and store it all inside one dataframe. We will also store the results inside a comunity objects data structure so that we can use the results later.

Table 6: Community Detection Comparison Across Subgraphs

Subgraph	Algorithm	Number of Communities	Modularity (Q)	Max Community Size
1	Fast Greedy	8	0.7267	56
1	Louvain	9	0.7586	49
1	Walktrap	10	0.7346	66
2	Fast Greedy	8	0.7086	44
2	Louvain	9	0.7090	47
2	Walktrap	20	0.6767	39
3	Fast Greedy	8	0.7685	45
3	Louvain	10	0.7886	35
3	Walktrap	10	0.7818	38
4	Fast Greedy	7	0.7436	40
4	Louvain	8	0.7397	38
4	Walktrap	12	0.7357	38

Looking at the results, we see that Walktrap generally detects more communities than the other 2 algorithms. All of the algorithms have high modularity greater than 0.7, suggesting that they found clear communities. It seems that subgraph 3 gets the highest modularity values compared to the other subgraphs across all three algorithms. With this we will dive deeper into observing subgraph 3 by visualize our network again.

When viewing our visualization of subgraph 3 with a KK layout, we can see that our previous observations matched what we are seeing here. In that we noticed that we have two significant/larger clusters. And that in those larger clusters, we have many subclusters within them.

Adjacency Matrix

Table 7: Comparison of Community Detection Algorithms Across Subgraphs

Subgraph	Algorithm	Number of Communities	Modularity (Q)	Largest Community Size
1	Fast Greedy	8	0.7267	56
1	Louvain	9	0.7586	49
1	Walktrap	10	0.7346	66
2	Fast Greedy	8	0.7086	44
2	Louvain	9	0.7090	47
2	Walktrap	20	0.6767	39
3	Fast Greedy	8	0.7685	45
3	Louvain	10	0.7886	35
3	Walktrap	10	0.7818	38
4	Fast Greedy	7	0.7436	40
4	Louvain	8	0.7397	38
4	Walktrap	12	0.7357	38

Adjacency Matrix

The adjacency matrix of each 200-node subgraph was examined to identify structural patterns in the Google web graph. Each entry in the matrix represents the presence of a directed hyperlink between two pages. In the original node ordering, edges appear scattered with no discernible pattern, reflecting the arbitrary order in which nodes were sampled. To reveal underlying structure, nodes were reordered by Louvain community membership. After reordering, clear diagonal block patterns emerged, particularly in Subgraph 4 which contained 1,608 edges. Each block along the diagonal represents a group of web pages that densely hyperlink

to one another, likely corresponding to topically related pages. The sparsity off the diagonal indicates that pages from different communities rarely link directly to each other. This visual confirmation aligns with the community detection results, supporting the presence of meaningful cluster structure within the Google web graph.

Reordering Nodes for Community Detection

As we see above, all of the nodes are in one connected component. As such, this step is essentially the same as above. We will reorder the adjacency matrices for all of the remaining subgraphs as well.

After reordering by community for each algorithm, we can see that the nodes form multiple boxes. This represents communities where nodes are highly interconnected. The adjacency matrix also happens to be mirrored across the y=x line, and that's because we used the undirected graph. We chose to use the undirected graph instead of the directed graph because that is the graph the algorithms used to detect the communities. We notice that there are two main shapes of dense boxes or relatively more sparse L shapes. Additionally, we see that Walktrap has more smaller boxes compared to Fast Greedy and Louvain.

Results

To address the extreme sparsity of the full web graph, random walk sampling was used to generate four representative subgraphs. For each sample, a walk began from a randomly selected seed node and proceeded for 500 steps through the network, and the first 200 unique nodes visited were used to construct an induced subgraph. This approach allowed us to capture locally connected regions of the web with meaningful hyperlink structure.

Across all four sampled subgraphs, the networks exhibit clustered structures connected by a small number of bridging nodes, rather than forming a single densely connected component. This indicates that webpages tend to organize into tightly linked groups with relatively few connections between different clusters. Interestingly, Subgraph 1 shows a more fragmented structure consisting of several dense clusters connected by sparse bridge nodes, while subgraph 4 reveal two main communities connected by narrow linking paths. These patterns suggest that webpages are organized into internally linked sections, with a small number of bridging pages forming paths between different parts of the network.

Connectivity varies depending on where the random walk occurs in the graph, with mean node degrees ranging from approximately 8.4 to 16.1, and Subgraph 4 representing the most densely connected region. Visualizations using Fruchterman–Reingold, Kamada–Kawai, and MDS layouts consistently highlight community clustering, branching structures, and intermediary nodes connecting separate regions of the graph.

Community detection further supports these structural insights. The Walktrap algorithm generally detects more communities than Louvain or Fast-Greedy, indicating smaller subcommunities within larger clusters, while all algorithms produce high modularity scores (around 0.7 or higher). Reordering adjacency matrices by Louvain community membership reveals clear block-diagonal patterns, confirming that most edges occur within communities and only a few nodes link different clusters. Occasional L-shaped patterns represent hub-like webpages connecting multiple communities.

Overall, these results demonstrate that random walk sampling effectively captured locally dense regions of the web graph while preserving key structural properties, including modularity, hierarchical communities, and hub-mediated connectivity.

Conclusion

This analysis explored the structural organization of the early internet by examining the Google web graph from 2002 using social network analysis techniques. Because the full network is extremely large and sparse, random walk sampling was used to extract representative subgraphs that capture locally connected regions of the web. This approach allowed us to analyze meaningful hyperlink structures while maintaining computational feasibility.

Several key insights emerged from the analysis. First, the sampled networks consistently revealed a strong community-based structure, where webpages form tightly connected clusters with relatively few links connecting different groups. Network metrics such as reciprocity and transitivity suggest that webpages frequently reinforce connections within their own local neighborhoods, creating cohesive communities of related content. Visualizations across multiple layouts further highlighted these patterns, revealing clusters, branching paths, and intermediary nodes that connect different regions of the network.

Community detection algorithms reinforced these observations. High modularity scores across all tested methods indicate clear and stable community boundaries, while the larger number of communities detected by Walktrap suggests the presence of hierarchical subcommunities within broader clusters. Adjacency matrix reordering also visually confirmed this structure through clear block-diagonal patterns, demonstrating that hyperlinks are concentrated within topical clusters rather than distributed randomly across the network.

Together, these results highlight that the early web was not organized as a uniform or random network. Instead, it exhibited a modular and hierarchical architecture, where clusters of related webpages were connected through a smaller set of bridging pages and hubs. Understanding this structure helps explain how information flowed through the early internet and provides insight into how search engines such as Google could identify important pages and rank content effectively.

Future work could expand on this analysis by exploring larger sampled subgraphs, comparing different network sampling methods, or incorporating temporal web datasets to examine how the structure of the internet evolved over time. Additional analysis could also investigate centrality measures, link prediction, or topic-based clustering to further understand how influential webpages and communities shaped the development of the web.

Overall, this study demonstrates that even when analyzing sampled portions of a massive network, important structural properties of the early internet can still be observed, offering valuable insight into the foundations of the interconnected web that continues to support modern information systems and AI technologies.

References

Leskovec, J., Lang, K. J., Dasgupta, A., & Mahoney, M. W. (2009).
Community structure in large networks: Natural cluster sizes and the absence of large well-defined clusters.
Internet Mathematics, 6(1), 29–123.
Dataset retrieved from the Stanford Network Analysis Project (SNAP):
https://snap.stanford.edu/data/web-Google.html

Csardi, G., & Nepusz, T. (2022).
igraph: Network Analysis and Visualization in R.
Random walk documentation:
https://r.igraph.org/reference/random_walk.html

Csardi, G., & Nepusz, T. (2022).
Random walk on a graph — igraph R documentation.
https://igraph.org/r/html/1.3.1/random_walk.html

Myakushina, A. (2023).
Community Detection in Complex Networks.
University of Rochester Mathematics Honors Thesis.
https://www.sas.rochester.edu/mth/undergraduate/honorspaperspdfs/a_myakushina23.pdf

Appendix

All Code Implemenation

```
knitr::opts_chunk$set(
  echo = FALSE,   # hides code
  message = FALSE,
  warning = FALSE
)
#Loading and Preparing Dataset
library(igraph)

url = "https://snap.stanford.edu/data/web-Google.txt.gz"
download.file(url, destfile = "web-Google.txt.gz", mode = "wb")

edges = read.table(
  "web-Google.txt.gz",
  comment.char = "#",
)

graph = graph_from_data_frame(edges, directed=TRUE)
avg.degree= mean(degree(graph))
max.degree = max(degree(graph))
density = edge_density(graph)

Dataset.info= data.frame(
  Property = c(
    "Nodes", "Edges", "Directionality", "Weighted", "Self-loops", " Density",
    "Mean Degree","Max Degree"
  ),
  Value = c(
    vcount(graph),
    ecount(graph),
    ifelse(is_directed(graph), "Yes","No"),
    ifelse(is_weighted(graph), "Yes","No"),
    ifelse(any(which_loop(graph)), "Yes","No"),
    round(density,7),
    round(avg.degree,2),
    max.degree
  )
)
knitr::kable(Dataset.info,
             caption = "Basic Network Elements of the web-Google Network",
             align = c("l","l"))
set.seed(123)

sub.network = lapply(1:4, function(x){
  induced_subgraph(graph, sample(V(graph),200))
})
sub.network[[1]]
sub.network[[2]]
sub.network[[3]]
sub.network[[4]]
set.seed(123)
```

```r
subgraphs <- list()

for(i in 1:4){
  start_node <- sample(V(graph), 1)
  walk <- random_walk(graph,
                      start = start_node,
                      steps = 500,
                      mode = "all")
  nodes_sampled <- unique(walk)
  nodes_sampled <- nodes_sampled[1:200]
  subgraphs[[i]] <- induced_subgraph(graph, nodes_sampled)
}



subgraph.info = data.frame(
  Subgraph = paste("Subgraph", 1:4),
  Nodes = sapply(subgraphs, vcount),
  Edges = sapply(subgraphs, ecount)
)

knitr::kable(subgraph.info,
             caption = "Summary of Random Walk Subgraphs",
             align = c("l","r","r"))
deg.full = degree(graph)

bias.results = data.frame(
  Subgraph = character(),
  Nodes = numeric(),
  Edges = numeric(),
  Mean_Degree = numeric(),
  Median_Degree = numeric(),
  KS_p_value = numeric(),
  stringsAsFactors = FALSE
)

for (i in 1:4){
  deg.sample = degree(subgraphs[[i]])
  ks = ks.test(deg.sample, deg.full)
  bias.results = rbind(
    bias.results,
    data.frame(
      Subgraph = paste("Subgraph", i),
      Nodes = vcount(subgraphs[[i]]),
      Edges = ecount(subgraphs[[i]]),
      Mean_Degree = mean(deg.sample),
      Median_Degree = median(deg.sample),
      KS_p_value = ks$p.value
      )
    )
}
knitr::kable(bias.results,
             caption = "Sampling Bias Assessment of Random Walk Subgraphs",
```

```r
             col.names = c("Subgraph", "Nodes", "Edges",
                           "Mean Degree", "Median Degree", "KS P-Value"),
             align = c("l", "r", "r", "r", "r", "r"))
packages <- data.frame(
  Package = c("`igraph`", "`R.utils`", "`knitr`"),
  Purpose = c(
    "Graph construction, manipulation, metrics and visualization",
    "Decompressing the raw .gz dataset file",
    "Generating formatted tables and controlling RMarkdown output"
  )
)

knitr::kable(packages,
             caption = "R Packages Used",
             align = c("l", "l"))
par(mfrow = c(1,3))

plot(
  subgraphs[[1]],
  layout = layout_with_fr(subgraphs[[1]]),
  vertex.size = 4,
  vertex.label = NA,
  edge.arrow.size = 0.2,
  main = "Subgraph 1 - Fruchterman-Reingold"
)

plot(
  subgraphs[[1]],
  layout = layout_with_kk(subgraphs[[1]]),
  vertex.size = 4,
  vertex.label = NA,
  edge.arrow.size = 0.2,
  main = "Subgraph 1 - Kamada-Kawai"
)

plot(
  subgraphs[[1]],
  layout = layout_with_mds(subgraphs[[1]]),
  vertex.size = 4,
  vertex.label = NA,
  edge.arrow.size = 0.2,
  main = "Subgraph 1 - MDS"
)
par(mfrow = c(1,3))

plot(
  subgraphs[[2]],
  layout = layout_with_fr(subgraphs[[2]]),
  vertex.size = 4,
  vertex.label = NA,
  edge.arrow.size = 0.2,
  main = "Subgraph 2- Fruchterman-Reingold"
)
```

```r
plot(
  subgraphs[[2]],
  layout = layout_with_kk(subgraphs[[2]]),
  vertex.size = 4,
  vertex.label = NA,
  edge.arrow.size = 0.2,
  main = "Subgraph 2 - Kamada-Kawai"
)

plot(
  subgraphs[[2]],
  layout = layout_with_mds(subgraphs[[2]]),
  vertex.size = 4,
  vertex.label = NA,
  edge.arrow.size = 0.2,
  main = "Subgraph 2 - MDS"
)
par(mfrow = c(1,3))

plot(
  subgraphs[[3]],
  layout = layout_with_fr(subgraphs[[3]]),
  vertex.size = 4,
  vertex.label = NA,
  edge.arrow.size = 0.2,
  main = "Subgraph 3 - Fruchterman-Reingold"
)

plot(
  subgraphs[[3]],
  layout = layout_with_kk(subgraphs[[3]]),
  vertex.size = 4,
  vertex.label = NA,
  edge.arrow.size = 0.2,
  main = "Subgraph 3 - Kamada-Kawai"
)

plot(
  subgraphs[[3]],
  layout = layout_with_mds(subgraphs[[3]]),
  vertex.size = 4,
  vertex.label = NA,
  edge.arrow.size = 0.2,
  main = "Subgraph 3 - MDS"
)
par(mfrow = c(1,3))

plot(
  subgraphs[[4]],
  layout = layout_with_fr(subgraphs[[4]]),
  vertex.size = 4,
  vertex.label = NA,
  edge.arrow.size = 0.2,
```

```r
  main = "Subgraph 4 - Fruchterman-Reingold"
)

plot(
  subgraphs[[4]],
  layout = layout_with_kk(subgraphs[[4]]),
  vertex.size = 4,
  vertex.label = NA,
  edge.arrow.size = 0.2,
  main = "Subgraph 4 - Kamada-Kawai"
)

plot(
  subgraphs[[4]],
  layout = layout_with_mds(subgraphs[[4]]),
  vertex.size = 4,
  vertex.label = NA,
  edge.arrow.size = 0.2,
  main = "Subgraph 4 - MDS"
)
g <- subgraphs[[4]]
metrics_df <- data.frame(
  Metric = c(
    "Density",
    "Reciprocity",
    "Transitivity",
    "Mean Distance",
    "Diameter",
    "Dyad Census: Mutual",
    "Dyad Census: Asymmetric",
    "Dyad Census: Null",
    "Closeness Centralization",
    "Betweenness NaN Proportion",
    "Closeness NaN Proportion"
  ),
  Value = c(
    round(edge_density(g), 4),
    round(reciprocity(g), 4),
    round(transitivity(g), 4),
    round(mean_distance(g), 4),
    diameter(g, weights = NA),
    dyad_census(g)$mut,
    dyad_census(g)$asym,
    dyad_census(g)$null,
    round(centr_clo(g, mode = "all", normalized = TRUE)$centralization, 4),
    round(mean(is.nan(betweenness(g))), 4),
    round(mean(is.nan(closeness(g))), 4)
  )
)

knitr::kable(metrics_df, align = c("l", "r"))
par(mfrow = c(1,2),mar = c(5.1, 4.1, 4.1, 2.1))
hist(degree(g, mode = "in"),
```

```r
     xlim = c(0, 40),
     breaks = 1:vcount(g)-1,
     main = "In-Degree Histogram",
     xlab = "In-Degree")
hist(degree(g, mode = "out"),
     xlim = c(0, 40),
     breaks = 1:vcount(g)-1,
     main = "Out-Degree Histogram",
     xlab = "Out-Degree")
print(betweenness(g))
clr <- centr_clo(g, mode = "all", normalized = TRUE)
print(sort(clr$res, decreasing = TRUE, index.return = TRUE))
print(transitivity(g, type = "local"))
# convert all into undirected graphs
undirected_subgraphs <- vector("list", 4)
for (i in 1:4){
  undirected_subgraphs[[i]] <- as.undirected(subgraphs[[i]], mode="collapse")
}
# list to store all results
all_results <- list()
community_objects <- list()

for (i in 1:4) {
  g <- undirected_subgraphs[[i]]

  community_objects[[i]] <- list(
    fast_greedy = cluster_fast_greedy(g),
    louvain     = cluster_louvain(g),
    walktrap    = cluster_walktrap(g)
  )

  # Grab the objects we just saved to calculate stats
  objs <- community_objects[[i]]

  all_results[[i]] <- data.frame(
    Subgraph      = i,
    Algorithm     = c("Fast Greedy", "Louvain", "Walktrap"),
    N_communities = c(length(objs$fast_greedy), length(objs$louvain), length(objs$walktrap)),
    Modularity_Q  = round(c(modularity(objs$fast_greedy), modularity(objs$louvain), modularity(objs$wall
    Max_Comm_Size = c(max(sizes(objs$fast_greedy)), max(sizes(objs$louvain)), max(sizes(objs$walktrap))
  )
}

# combine list into one table
final_comparison <- do.call(rbind, all_results)



knitr::kable(final_comparison,
      caption = "Community Detection Comparison Across Subgraphs",
      col.names = c("Subgraph", "Algorithm", "Number of Communities",
                    "Modularity (Q)", "Max Community Size"))
g3 <- undirected_subgraphs[[3]]
```

```r
# compute layout once so all plots use the same node positions
L <- layout_with_kk(g3)

par(mfrow = c(1,3))

plot(community_objects[[3]]$fast_greedy, g3,
     layout = L,
     vertex.label = NA,
     vertex.size = 7,
     edge.color = "gray40",
     edge.width = 0.8,
     main = "Fast Greedy")

plot(community_objects[[3]]$louvain, g3,
     layout = L,
     vertex.label = NA,
     vertex.size = 7,
     edge.color = "gray40",
     edge.width = 0.8,
     main = "Louvain")

plot(community_objects[[3]]$walktrap, g3,
     layout = L,
     vertex.label = NA,
     vertex.size = 7,
     edge.color = "gray40",
     edge.width = 0.8,
     main = "Walktrap")

for(i in 1:4){
  adj_mat <- as_adjacency_matrix(subgraphs[[i]], sparse = FALSE)

  image(adj_mat,
        main = paste("Subgraph", i, "- Original Node Order"),
        col = c("white", "steelblue"),
        axes = FALSE)
}
# convert all into undirected graphs
undirected_subgraphs <- vector("list", 4)
for (i in 1:4){
  undirected_subgraphs[[i]] <- as.undirected(subgraphs[[i]])
}
# list to store all results
all_results <- list()

for (i in 1:4) {
  g <- undirected_subgraphs[[i]]

  # run algs
  fg   <- cluster_fast_greedy(g)
  louv <- cluster_louvain(g)
  walk <- cluster_walktrap(g)
```

```r
  # store into list
  all_results[[i]] <- data.frame(
    Subgraph      = i,
    Algorithm     = c("Fast Greedy", "Louvain", "Walktrap"),
    N_communities = c(length(fg), length(louv), length(walk)),
    Modularity_Q  = round(c(modularity(fg), modularity(louv), modularity(walk)), 4),
    Max_Comm_Size = c(max(sizes(fg)), max(sizes(louv)), max(sizes(walk)))
  )
}
# combine list into one table
final_comparison <- do.call(rbind, all_results)

knitr::kable(final_comparison,
      caption = "Comparison of Community Detection Algorithms Across Subgraphs",
      col.names = c("Subgraph",
                    "Algorithm",
                    "Number of Communities",
                    "Modularity (Q)",
                    "Largest Community Size"))
for(i in 1:4){
  adj_mat <- as_adjacency_matrix(subgraphs[[i]], sparse = FALSE)

  image(adj_mat,
        main = paste("Subgraph", i, "- Original Node Order"),
        sub = "Blue = hyperlink exists, White = no hyperlink",
        col = c("white", "steelblue"),
        axes = FALSE,
        xlab = "Node",
        ylab = "Node")
}
set.seed(42)
for(i in 1:4){
  adj_mat <- as_adjacency_matrix(subgraphs[[i]], sparse = FALSE)

  kc_louv <- cluster_louvain(undirected_subgraphs[[i]])
  node_order <- order(membership(kc_louv))
  adj_mat_reordered <- adj_mat[node_order, node_order]

  image(adj_mat_reordered,
        main = paste("Subgraph", i, "- Reordered by Community"),
        sub = "Blue = hyperlink exists, White = no hyperlink",
        col = c("white", "steelblue"),
        axes = FALSE,
        xlab = "Node (reordered)",
        ylab = "Node (reordered)")
}
for (i in 1:4) {
  A <- as_adjacency_matrix(undirected_subgraphs[[i]], sparse = FALSE)

  par(mfrow = c(1, 3))

  # loop through the algorithms
  alg_names <- names(community_objects[[i]])
```

```r
  for (alg in alg_names) {
    comm_obj <- community_objects[[i]][[alg]]

    ord <- order(membership(comm_obj))

    A_reordered <- A[ord, ord]

    image(A_reordered,
          main = paste("Sub", i, ":", alg),
          xlab = "Node (reordered)",
          ylab = "Node (reordered)",
          col=c("white", "steelblue"),
          axes = FALSE)
  }

  # reset plots for next one
  par(mfrow = c(1, 1))
}
par(mfrow = c(1,3))

plot(
  subgraphs[[1]],
  layout = layout_with_fr(subgraphs[[1]]),
  vertex.size = 4,
  vertex.label = NA,
  edge.arrow.size = 0.2,
  main = "Subgraph 1 - Fruchterman-Reingold"
)

plot(
  subgraphs[[1]],
  layout = layout_with_kk(subgraphs[[1]]),
  vertex.size = 4,
  vertex.label = NA,
  edge.arrow.size = 0.2,
  main = "Subgraph 1 - Kamada-Kawai"
)

plot(
  subgraphs[[1]],
  layout = layout_with_mds(subgraphs[[1]]),
  vertex.size = 4,
  vertex.label = NA,
  edge.arrow.size = 0.2,
  main = "Subgraph 1 - MDS"
)
par(mfrow = c(1,3))

plot(
  subgraphs[[2]],
  layout = layout_with_fr(subgraphs[[2]]),
  vertex.size = 4,
  vertex.label = NA,
```

```r
  edge.arrow.size = 0.2,
  main = "Subgraph 2- Fruchterman-Reingold"
)

plot(
  subgraphs[[2]],
  layout = layout_with_kk(subgraphs[[2]]),
  vertex.size = 4,
  vertex.label = NA,
  edge.arrow.size = 0.2,
  main = "Subgraph 2 - Kamada-Kawai"
)

plot(
  subgraphs[[2]],
  layout = layout_with_mds(subgraphs[[2]]),
  vertex.size = 4,
  vertex.label = NA,
  edge.arrow.size = 0.2,
  main = "Subgraph 2 - MDS"
)
par(mfrow = c(1,3))

plot(
  subgraphs[[3]],
  layout = layout_with_fr(subgraphs[[3]]),
  vertex.size = 4,
  vertex.label = NA,
  edge.arrow.size = 0.2,
  main = "Subgraph 3 - Fruchterman-Reingold"
)

plot(
  subgraphs[[3]],
  layout = layout_with_kk(subgraphs[[3]]),
  vertex.size = 4,
  vertex.label = NA,
  edge.arrow.size = 0.2,
  main = "Subgraph 3 - Kamada-Kawai"
)

plot(
  subgraphs[[3]],
  layout = layout_with_mds(subgraphs[[3]]),
  vertex.size = 4,
  vertex.label = NA,
  edge.arrow.size = 0.2,
  main = "Subgraph 3 - MDS"
)
par(mfrow = c(1,3))

plot(
  subgraphs[[4]],
```

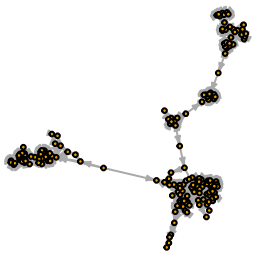
```r
  layout = layout_with_fr(subgraphs[[4]]),
  vertex.size = 4,
  vertex.label = NA,
  edge.arrow.size = 0.2,
  main = "Subgraph 4 - Fruchterman-Reingold"
)

plot(
  subgraphs[[4]],
  layout = layout_with_kk(subgraphs[[4]]),
  vertex.size = 4,
  vertex.label = NA,
  edge.arrow.size = 0.2,
  main = "Subgraph 4 - Kamada-Kawai"
)

plot(
  subgraphs[[4]],
  layout = layout_with_mds(subgraphs[[4]]),
  vertex.size = 4,
  vertex.label = NA,
  edge.arrow.size = 0.2,
  main = "Subgraph 4 - MDS"
)
print(betweenness(g))
clr <- centr_clo(g, mode = "all", normalized = TRUE)
print(sort(clr$res, decreasing = TRUE, index.return = TRUE))
print(transitivity(g, type = "local"))
g3 <- undirected_subgraphs[[3]]

# compute layout once so all plots use the same node positions
L <- layout_with_kk(g3)

par(mfrow = c(1,3))

plot(community_objects[[3]]$fast_greedy, g3,
     layout = L,
     vertex.label = NA,
     vertex.size = 7,
     edge.color = "gray40",
     edge.width = 0.8,
     main = "Fast Greedy")

plot(community_objects[[3]]$louvain, g3,
     layout = L,
     vertex.label = NA,
     vertex.size = 7,
     edge.color = "gray40",
     edge.width = 0.8,
     main = "Louvain")

plot(community_objects[[3]]$walktrap, g3,
     layout = L,
```

```r
     vertex.label = NA,
     vertex.size = 7,
     edge.color = "gray40",
     edge.width = 0.8,
     main = "Walktrap")

for(i in 1:4){
  adj_mat <- as_adjacency_matrix(subgraphs[[i]], sparse = FALSE)

  image(adj_mat,
        main = paste("Subgraph", i, "- Original Node Order"),
        col = c("white", "steelblue"),
        axes = FALSE)
}
# convert all into undirected graphs
undirected_subgraphs <- vector("list", 4)
for (i in 1:4){
  undirected_subgraphs[[i]] <- as.undirected(subgraphs[[i]])
}
# list to store all results
all_results <- list()

for (i in 1:4) {
  g <- undirected_subgraphs[[i]]

  # run algs
  fg   <- cluster_fast_greedy(g)
  louv <- cluster_louvain(g)
  walk <- cluster_walktrap(g)

  # store into list
  all_results[[i]] <- data.frame(
    Subgraph      = i,
    Algorithm     = c("Fast Greedy", "Louvain", "Walktrap"),
    N_communities = c(length(fg), length(louv), length(walk)),
    Modularity_Q  = round(c(modularity(fg), modularity(louv), modularity(walk)), 4),
    Max_Comm_Size = c(max(sizes(fg)), max(sizes(louv)), max(sizes(walk)))
  )
}
for(i in 1:4){
  adj_mat <- as_adjacency_matrix(subgraphs[[i]], sparse = FALSE)

  image(adj_mat,
        main = paste("Subgraph", i, "- Original Node Order"),
        sub = "Blue = hyperlink exists, White = no hyperlink",
        col = c("white", "steelblue"),
        axes = FALSE,
        xlab = "Node",
        ylab = "Node")
}
set.seed(42)
for(i in 1:4){
  adj_mat <- as_adjacency_matrix(subgraphs[[i]], sparse = FALSE)
```

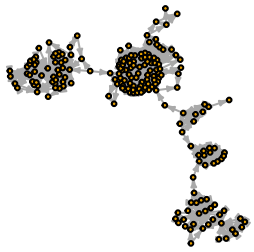
```r
  kc_louv <- cluster_louvain(undirected_subgraphs[[i]])
  node_order <- order(membership(kc_louv))
  adj_mat_reordered <- adj_mat[node_order, node_order]

  image(adj_mat_reordered,
        main = paste("Subgraph", i, "- Reordered by Community"),
        sub = "Blue = hyperlink exists, White = no hyperlink",
        col = c("white", "steelblue"),
        axes = FALSE,
        xlab = "Node (reordered)",
        ylab = "Node (reordered)")
}
for (i in 1:4) {
  A <- as_adjacency_matrix(undirected_subgraphs[[i]], sparse = FALSE)

  par(mfrow = c(1, 3))

  # loop through the algorithms
  alg_names <- names(community_objects[[i]])

  for (alg in alg_names) {
    comm_obj <- community_objects[[i]][[alg]]

    ord <- order(membership(comm_obj))

    A_reordered <- A[ord, ord]

    image(A_reordered,
          main = paste("Sub", i, ":", alg),
          xlab = "Node (reordered)",
          ylab = "Node (reordered)",
          col=c("white", "steelblue"),
          axes = FALSE)
  }

  # reset plots for next one
  par(mfrow = c(1, 1))
}
```

Subgraph 1 Visual

Subgraph 1 − Fruchterman−Rein



Subgraph 1 − Kamada−Kawai

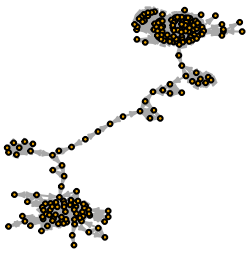


Subgraph 1 − MDS

Subgraph 2 Visuals

Subgraph 2− Fruchterman−Reing

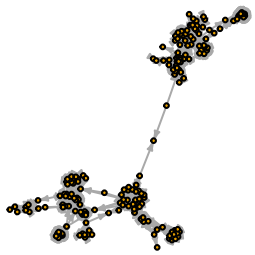
Subgraph 2 − Kamada−Kawai



Subgraph 2 − MDS

Subgraph 3 Visuals

Subgraph 3 − Fruchterman−Rein

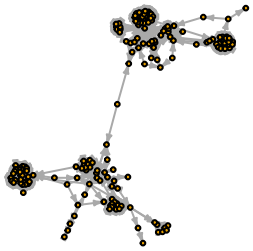


Subgraph 3 − Kamada−Kawai

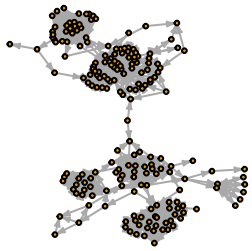
Subgraph 3 − MDS

Subgraph 4 Visuals

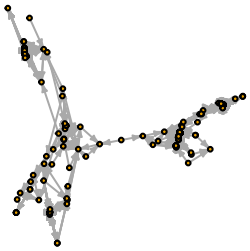
Subgraph 4 − Fruchterman−Rein



Subgraph 4 − Kamada−Kawai



Subgraph 4 − MDS



Betweenness Results

```
##       223236       213338        78244       147159       599130       527425
## 8.600297e+03 2.404473e+02 9.510970e+02 1.563015e+03 1.177102e+04 4.991667e+01
##       366369       369637       463481       495374       787300        92139
## 3.800000e+00 3.800000e+00 1.154160e+03 3.800000e+00 9.829042e+03 2.837795e+03
##       814716       218469       102796       347316       838750         5850
## 3.198690e+01 1.542489e+01 7.990952e+01 2.792489e+01 5.293712e+01 1.434341e+01
##        10261        58112        61175       129662       303595       405495
## 1.645452e+01 9.427778e+01 1.434341e+01 9.427778e+01 1.645452e+01 3.116944e+03
##       408499       599129       629001       669187       818894        48881
## 1.828785e+01 2.146033e+03 1.645452e+01 3.944444e+00 1.434341e+01 9.844000e+03
##       541901       351897       421091       425667       822292         9715
## 1.000000e+00 5.450000e+01 4.462267e+03 5.469048e+00 1.945590e+03 0.000000e+00
##        38629        77015        99844       148507       166327       244941
## 6.111111e-01 2.948728e+03 0.000000e+00 2.269106e+01 2.269106e+01 6.111111e-01
##       274872       304932       306026       357788       515058       589787
```

```
## 6.111111e-01 6.111111e-01 0.000000e+00 6.111111e-01 6.111111e-01 2.269106e+01
##       598864       604214       619636       624116       676284       696680
## 0.000000e+00 1.999106e+01 2.269106e+01 2.269106e+01 2.269106e+01 2.269106e+01
##       703794       744562       915782       861111        10723       316281
## 2.269106e+01 4.944444e+00 0.000000e+00 1.434341e+01 3.333333e-01 0.000000e+00
##       459215       481534       711344       862211       130153       302068
## 0.000000e+00 0.000000e+00 0.000000e+00 0.000000e+00 0.000000e+00 0.000000e+00
##       321160       733876       774138       869884       894841       864583
## 6.000000e-01 6.000000e-01 6.000000e-01 6.000000e-01 7.473147e+02 0.000000e+00
##        21049       205274       359900       407829       717470       731551
## 3.571429e-02 3.571429e-02 3.571429e-02 3.571429e-02 3.571429e-02 3.571429e-02
##       376719        29035       527888       706531       733467       418056
## 0.000000e+00 1.434341e+01 1.434341e+01 1.434341e+01 1.186026e+03 1.434341e+01
##        35152        53616        64989       127539       337809       377032
## 1.155285e+01 1.833995e+03 1.155285e+01 1.155285e+01 1.155285e+01 4.286237e+01
##       398209       594026       710290       867941       870959       487630
## 1.155285e+01 1.155285e+01 1.155285e+01 1.155285e+01 1.155285e+01 0.000000e+00
##       462857       273168       770891        75783        85406       435217
## 0.000000e+00 9.873425e+03 4.875000e+02 8.299339e+01 8.299339e+01 8.299339e+01
##       441612       506789       545842       702627       737494       832513
## 8.299339e+01 8.299339e+01 8.299339e+01 2.268322e+02 8.299339e+01 1.269707e+02
##       880108       432856        55159       573537       429945        72762
## 8.299339e+01 0.000000e+00 0.000000e+00 1.734650e+03 0.000000e+00 1.183018e+03
##       381879       769891       882352       707813       372733       887429
## 3.309288e+00 1.267838e+01 0.000000e+00 2.284674e+02 0.000000e+00 3.374167e+02
##        92993       350612       822238        98007       279506       780437
## 5.664854e+00 7.692308e-02 7.692308e-02 1.434341e+01 4.532118e+02 5.230000e+02
##       110018       116848       145806       227801       566222       566463
## 0.000000e+00 0.000000e+00 3.571429e-02 3.571429e-02 3.571429e-02 3.571429e-02
##       894224       288084       481627       512625       378722       228608
## 3.571429e-02 0.000000e+00 0.000000e+00 0.000000e+00 0.000000e+00 2.711783e+00
##       434026       820447       350376       187805       190643       836130
## 2.711783e+00 2.711783e+00 1.980000e+02 0.000000e+00 2.782626e+02 4.417952e+02
##       199181       226406       594844       632012       239220       251670
## 0.000000e+00 0.000000e+00 0.000000e+00 3.571429e-02 1.666667e-01 7.916667e+01
##       398571       261148       516181       276527       278785       502796
## 1.556935e+01 0.000000e+00 0.000000e+00 5.300000e+01 3.479634e+01 6.400000e+01
##       716299       408593       699352       792504       293854       353529
## 0.000000e+00 8.747619e+01 0.000000e+00 4.676190e+02 0.000000e+00 2.034690e+02
##       563896       317022       416918       533571       411906       403852
## 0.000000e+00 0.000000e+00 7.692308e-02 7.692308e-02 0.000000e+00 0.000000e+00
##       366823       415699       353036       368597       746103       411760
## 0.000000e+00 0.000000e+00 0.000000e+00 1.666667e-01 0.000000e+00 2.656667e+02
##       824946       418534       782470       538370       851895       549765
## 1.852500e+02 0.000000e+00 0.000000e+00 4.000000e-01 0.000000e+00 0.000000e+00
##       873367       475631       697355       504051       735047       599946
## 0.000000e+00 3.571429e-02 1.000000e+00 1.666667e-01 0.000000e+00 1.516667e+01
##       623268       626894       634510       640913       648220       659013
## 0.000000e+00 1.263333e+02 0.000000e+00 8.755265e+01 0.000000e+00 5.628427e+02
##       679947       705058       904987       560064       812833       227012
## 1.666667e-01 1.666667e-01 0.000000e+00 1.475667e+02 1.155265e+01 4.875000e+02
##       566018         2704
## 0.000000e+00 0.000000e+00
```

Closeness Centrality Results

```
## $x
##   [1] 0.2646277 0.2614980 0.2567742 0.2515803 0.2444717 0.2335681 0.2332943
##   [8] 0.2330211 0.2256236 0.2246050 0.2246050 0.2246050 0.2246050 0.2246050
##  [15] 0.2246050 0.2246050 0.2246050 0.2243517 0.2223464 0.2218506 0.2213571
##  [22] 0.2213571 0.2213571 0.2213571 0.2213571 0.2213571 0.2211111 0.2208657
##  [29] 0.2208657 0.2208657 0.2208657 0.2208657 0.2206208 0.2198895 0.2198895
##  [36] 0.2194046 0.2191630 0.2191630 0.2191630 0.2189219 0.2189219 0.2189219
##  [43] 0.2186813 0.2184413 0.2182018 0.2182018 0.2182018 0.2179628 0.2165397
##  [50] 0.2163043 0.2160695 0.2160695 0.2158351 0.2158351 0.2158351 0.2156013
##  [57] 0.2156013 0.2156013 0.2156013 0.2151351 0.2139785 0.2121535 0.2112527
##  [64] 0.2108051 0.2108051 0.2108051 0.2108051 0.2108051 0.2108051 0.2108051
##  [71] 0.2108051 0.2103594 0.2103594 0.2103594 0.2101373 0.2075078 0.2068607
##  [78] 0.2068607 0.2064315 0.2041026 0.2041026 0.2041026 0.2032686 0.2030612
##  [85] 0.2030612 0.2030612 0.2030612 0.2028542 0.2028542 0.2028542 0.2026477
##  [92] 0.2022358 0.2022358 0.2020305 0.2012133 0.1991992 0.1968348 0.1966403
##  [99] 0.1952895 0.1926428 0.1900669 0.1891635 0.1879131 0.1859813 0.1849442
## [106] 0.1844300 0.1827365 0.1820677 0.1797651 0.1791179 0.1791179 0.1786355
## [113] 0.1786355 0.1786355 0.1786355 0.1786355 0.1786355 0.1786355 0.1786355
## [120] 0.1786355 0.1781558 0.1781558 0.1773619 0.1770463 0.1768889 0.1768889
## [127] 0.1768889 0.1768889 0.1768889 0.1768889 0.1768889 0.1768889 0.1751761
## [134] 0.1747147 0.1745614 0.1731941 0.1715517 0.1714040 0.1714040 0.1712565
## [141] 0.1712565 0.1712565 0.1712565 0.1712565 0.1711092 0.1711092 0.1711092
## [148] 0.1711092 0.1711092 0.1706690 0.1706690 0.1706690 0.1677909 0.1677909
## [155] 0.1662490 0.1661102 0.1661102 0.1661102 0.1658333 0.1655574 0.1655574
## [162] 0.1655574 0.1655574 0.1654198 0.1654198 0.1654198 0.1651452 0.1651452
## [169] 0.1651452 0.1651452 0.1651452 0.1651452 0.1651452 0.1651452 0.1647351
## [176] 0.1643270 0.1624490 0.1624490 0.1624490 0.1624490 0.1624490 0.1592000
## [183] 0.1574367 0.1574367 0.1571880 0.1564465 0.1551052 0.1521407 0.1519084
## [190] 0.1519084 0.1498494 0.1438901 0.1423462 0.1363014 0.1363014 0.1363014
## [197] 0.1363014 0.1363014 0.1353741 0.1344595
##
## $ix
##   [1]   5  98  30  11   1  35 150  38   4  40  41  48  51  52  53  54  55  50
##  [19]  56   3  37  42  43  44  46  47  33  36  39  45  49  57  32  96 153  59
##  [37]  61  62  63 151 165 166 164  60  72 137 152  65 125 144  15  17 138 139
##  [55] 140  64  97 176 177 188  79  16   9   2  31  67  68  69  70  83 183 110
##  [73] 167 189  14 126  13 162  34   7   8  10  86 146 147 169 170 111 145 187
##  [91]  12 158 196 178 160 159 195 114 192  24 155  26  71 143 112 163 190 174
## [109] 186  90 156  85  87  88  89  91  92  93  94  95 157 168 106 108 100 101
## [127] 102 103 104 105 107 109 154 115 116 120 200 121 123 136 161 171 173 185
## [145] 117 128 179 180 181 113 134 135  20  22  25  19  23  27 118  80  81  82
## [163] 124  18  21  29  58  73  74  75  76  77  78 148  84 182 129 130 131 132
## [181] 133  66  99 198 197 141 175 122 127 142   6  28 191 149 172 184 193 194
## [199] 199 119
```

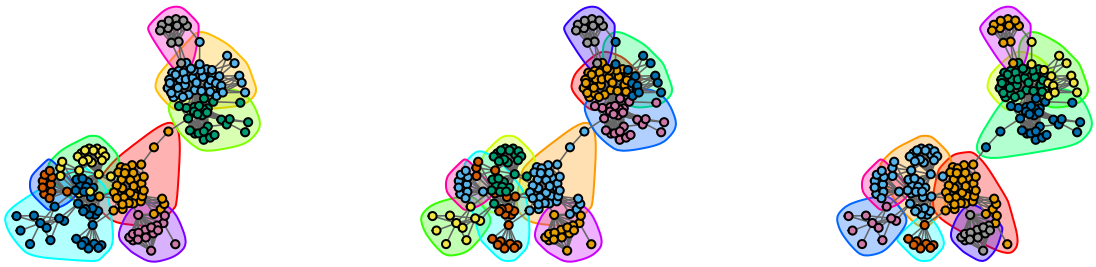
Local Transitivity Results

```
##    223236    213338     78244    147159    599130    527425    366369    369637
## 0.1733333 0.5000000 0.0000000 0.3250000 0.2302885 0.6666667 0.6545455 0.6545455
##    463481    495374    787300     92139    814716    218469    102796    347316
## 0.5454545 0.6545455 0.3333333 0.6068376 0.4666667 0.3333333 0.6666667 0.5277778
##    838750      5850     10261     58112     61175    129662    303595    405495
## 0.7500000 0.9090909 0.8000000 0.3359788 0.9090909 0.3359788 0.8000000 0.3540230
```

```
##    408499    599129    629001    669187    818894     48881    541901    351897
## 0.7426471 0.3544974 0.8000000 0.6153846 0.9090909 0.0000000 0.8333333 0.6666667
##    421091    425667    822292      9715     38629     77015     99844    148507
## 0.2363636 0.7000000 0.4761905 1.0000000 0.9601449 0.5832148 1.0000000 0.6156156
##    166327    244941    274872    304932    306026    357788    515058    589787
## 0.6156156 0.9601449 0.9601449 0.9601449 1.0000000 0.9601449 0.9601449 0.6156156
##    598864    604214    619636    624116    676284    696680    703794    744562
## 1.0000000 0.6365079 0.6156156 0.6156156 0.6156156 0.6156156 0.6156156 0.8333333
##    915782    861111     10723    316281    459215    481534    711344    862211
## 1.0000000 0.8611111 0.9666667 1.0000000 1.0000000 1.0000000 1.0000000 1.0000000
##    130153    302068    321160    733876    774138    869884    894841    864583
## 1.0000000       NaN 0.8571429 0.8571429 0.8571429 0.8571429 0.7252747 1.0000000
##     21049    205274    359900    407829    717470    731551    376719     29035
## 0.9777778 0.9777778 0.9777778 0.9777778 0.9777778 0.9777778 1.0000000 0.9242424
##    527888    706531    733467    418056     35152     53616     64989    127539
## 0.9242424 0.9242424 0.0000000 0.6666667 0.9142857 0.8000000 0.9142857 0.9142857
##    337809    377032    398209    594026    710290    867941    870959    487630
## 0.9142857 0.8000000 0.9142857 0.9142857 0.9142857 0.9142857 0.9142857 1.0000000
##    462857    273168    770891     75783     85406    435217    441612    506789
## 1.0000000 0.1666667 0.0000000 0.5960591 0.5960591 0.5960591 0.5960591 0.5960591
##    545842    702627    737494    832513    880108    432856     55159    573537
## 0.5960591 0.5563218 0.5960591 0.5563218 0.5960591 1.0000000 1.0000000 0.0000000
##    429945     72762    381879    769891    882352    707813    372733    887429
## 1.0000000 0.7333333 0.9428571 0.9560440 1.0000000 0.0000000 1.0000000 0.3333333
##     92993    350612    822238     98007    279506    780437    110018    116848
## 0.9743590 0.9848485 0.9871795 0.9242424 0.5555556 0.3333333 1.0000000 1.0000000
##    145806    227801    566222    566463    894224    288084    481627    512625
## 0.9523810 0.9523810 0.9523810 0.9523810 0.9523810 1.0000000 1.0000000 1.0000000
##    378722    228608    434026    820447    350376    187805    190643    836130
## 1.0000000 0.9523810 0.9523810 0.9523810 0.0000000 1.0000000 0.0000000 0.7500000
##    199181    226406    594844    632012    239220    251670    398571    261148
## 1.0000000 1.0000000 1.0000000 0.9777778 0.0000000 0.5000000 0.8681319 1.0000000
##    516181    276527    278785    502796    716299    408593    699352    792504
## 1.0000000 0.0000000 0.0000000 0.0000000       NaN 0.5000000 1.0000000 0.5000000
##    293854    353529    563896    317022    416918    533571    411906    403852
## 1.0000000 0.4666667 1.0000000 1.0000000 0.9890110 0.9890110 1.0000000 1.0000000
##    366823    415699    353036    368597    746103    411760    824946    418534
## 1.0000000 1.0000000 1.0000000 0.0000000 1.0000000 0.0000000 0.3333333 1.0000000
##    782470    538370    851895    549765    873367    475631    697355    504051
## 1.0000000 0.8333333 1.0000000 1.0000000 1.0000000 0.8333333 0.8333333 0.0000000
##    735047    599946    623268    626894    634510    640913    648220    659013
## 1.0000000 0.0000000 1.0000000 0.1666667 1.0000000 0.0000000       NaN 0.8333333
##    679947    705058    904987    560064    812833    227012    566018      2704
## 0.0000000 0.0000000 1.0000000 0.3333333 0.0000000 0.0000000       NaN       NaN
```

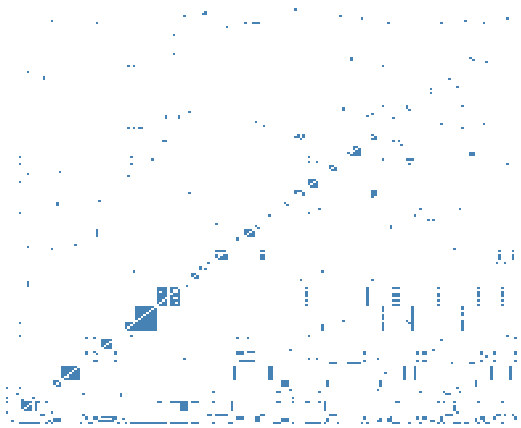
Subgraph 3 Community Detection Visual

Fast Greedy **Louvain** **Walktrap**

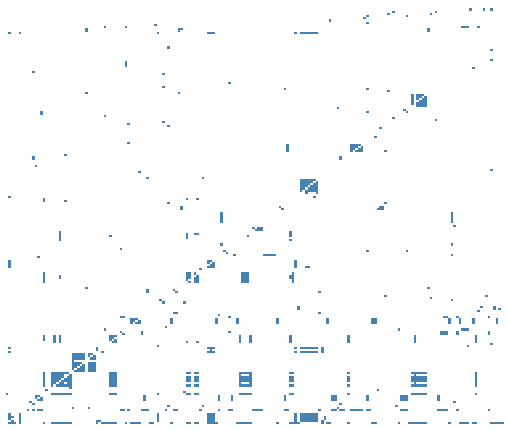


All Adjacency Matrix Visuals

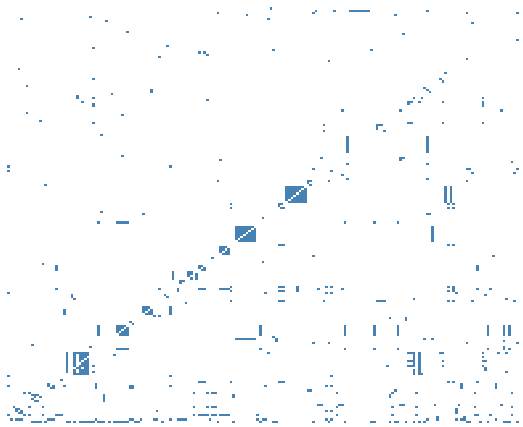
Subgraph 1 − Original Node Order



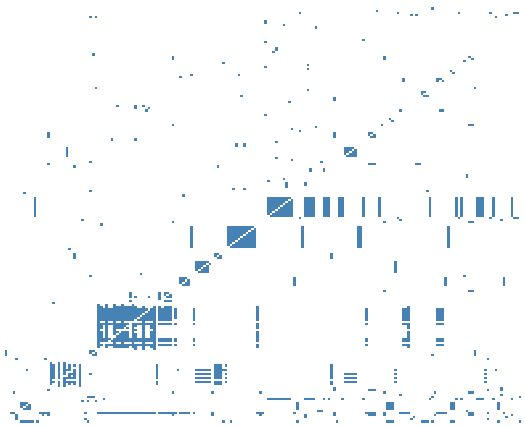
Subgraph 2 − Original Node Order



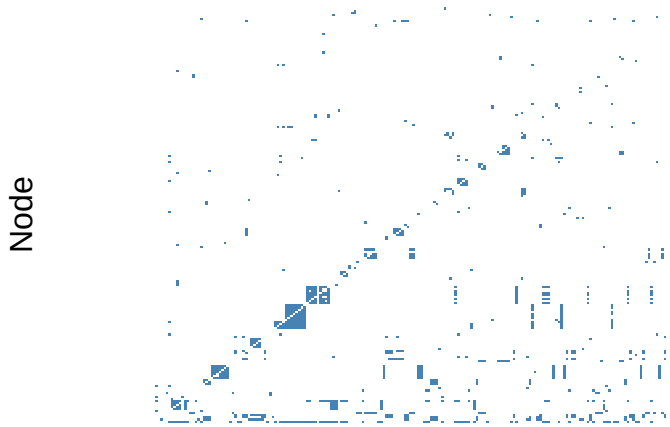
Subgraph 3 − Original Node Order



Subgraph 4 − Original Node Order

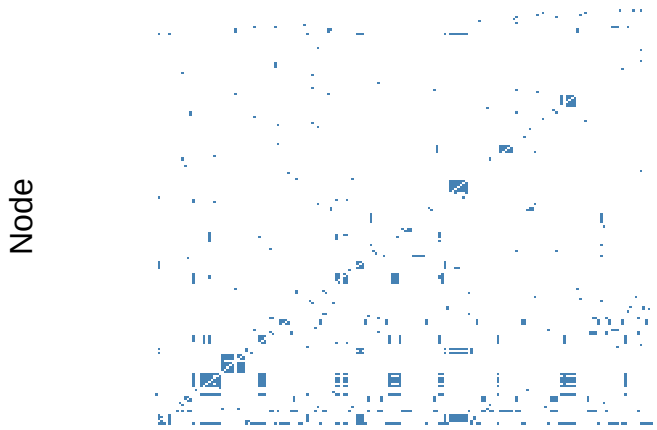


Subgraph 1 − Original Node Order



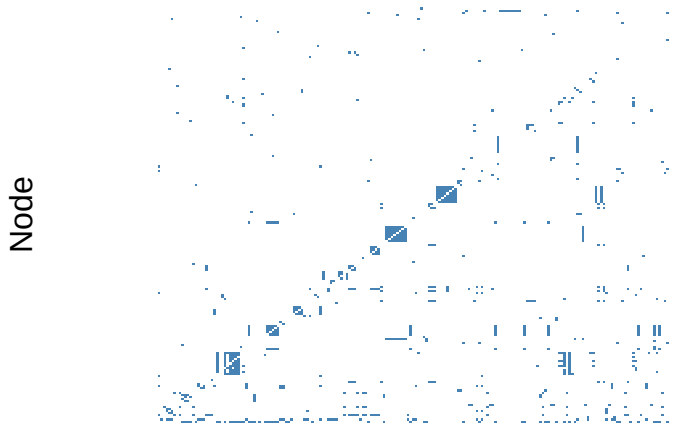
Blue = hyperlink exists, White = no hyperlink

Subgraph 2 − Original Node Order



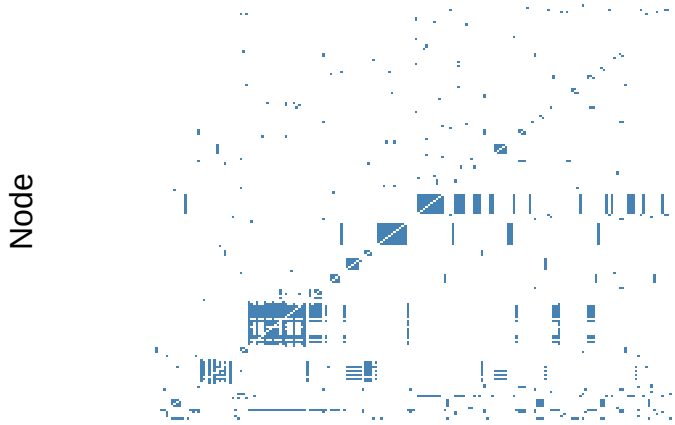
Blue = hyperlink exists, White = no hyperlink

Subgraph 3 − Original Node Order

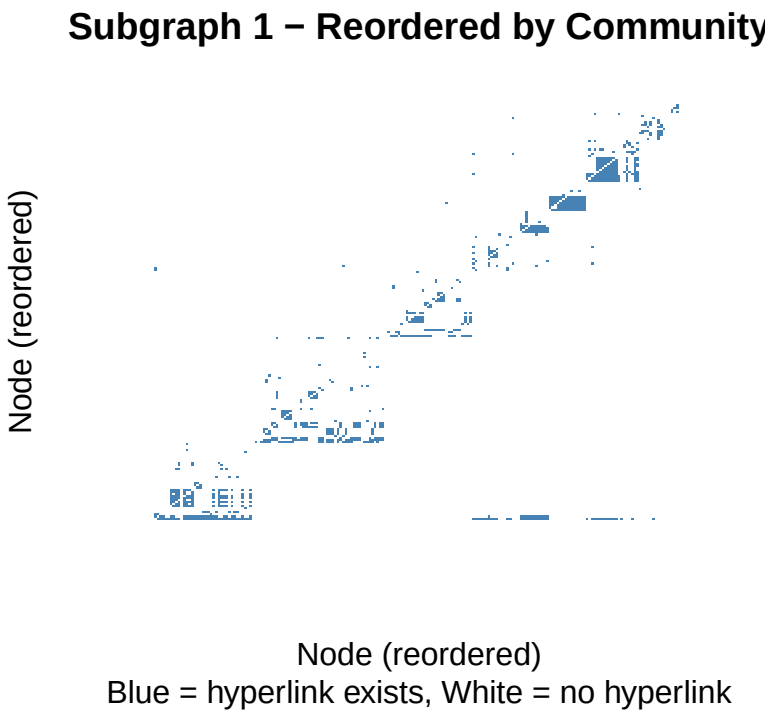


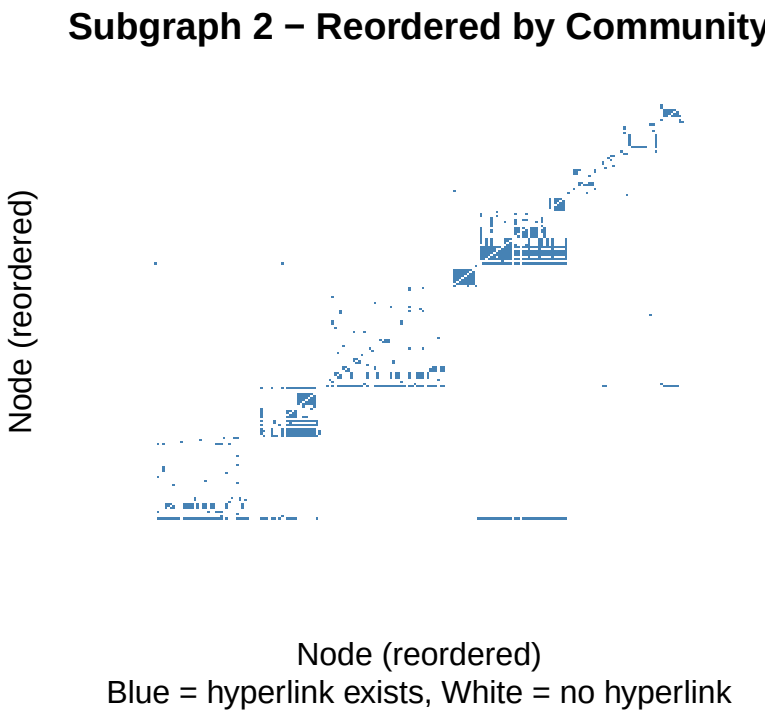
Blue = hyperlink exists, White = no hyperlink

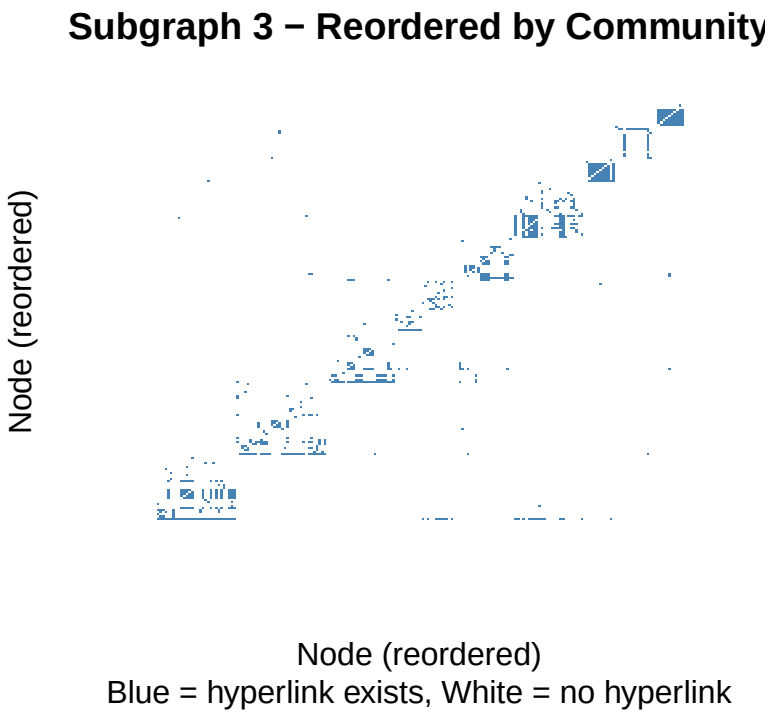
Subgraph 4 − Original Node Order

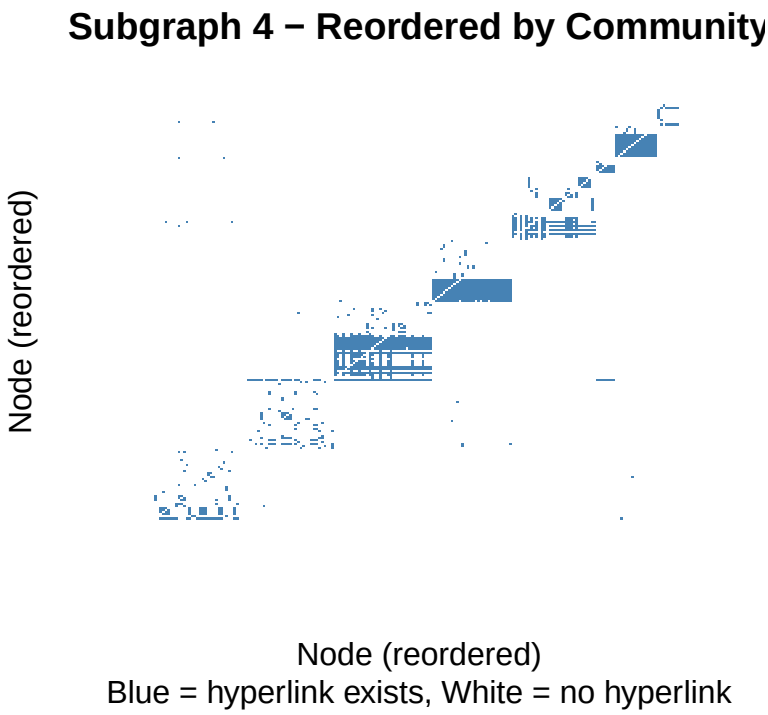


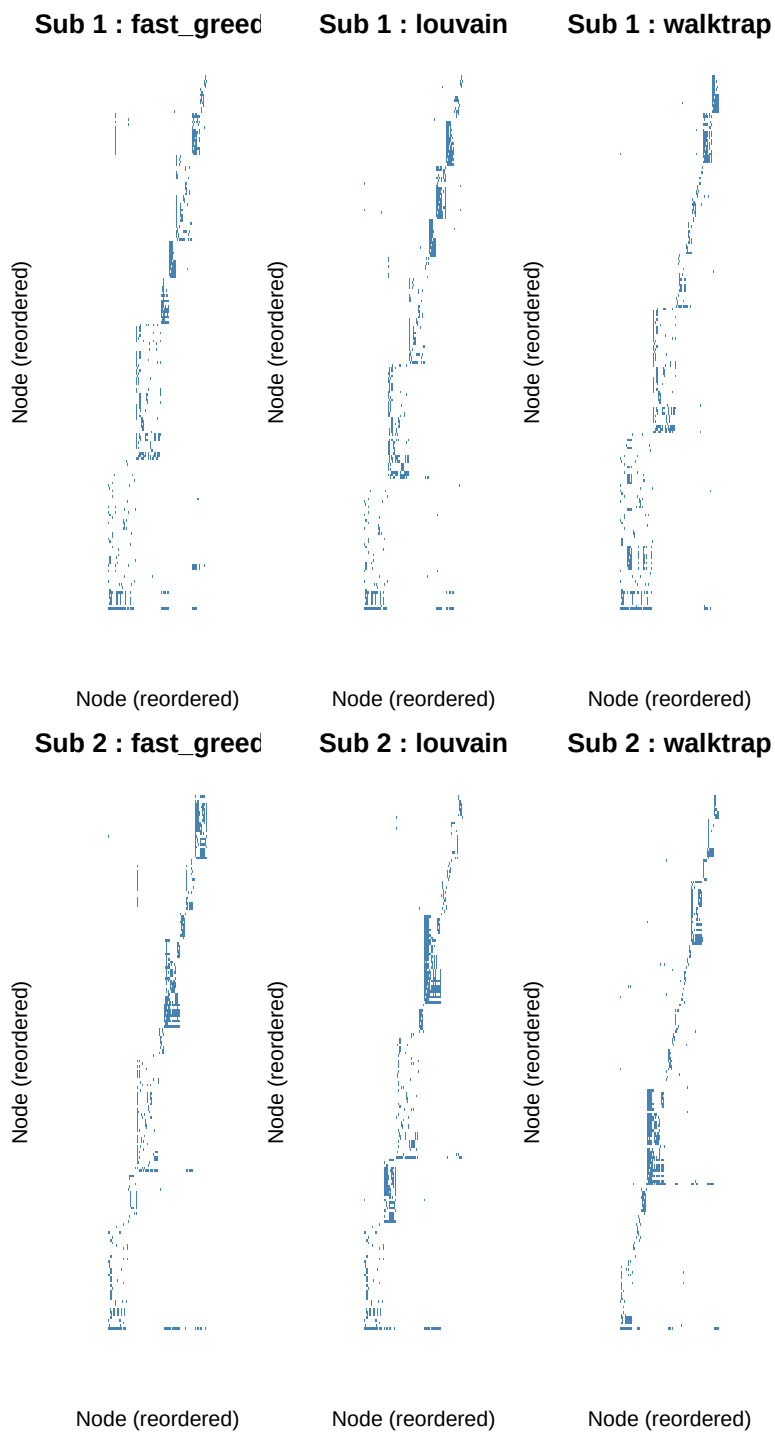
Blue = hyperlink exists, White = no hyperlink

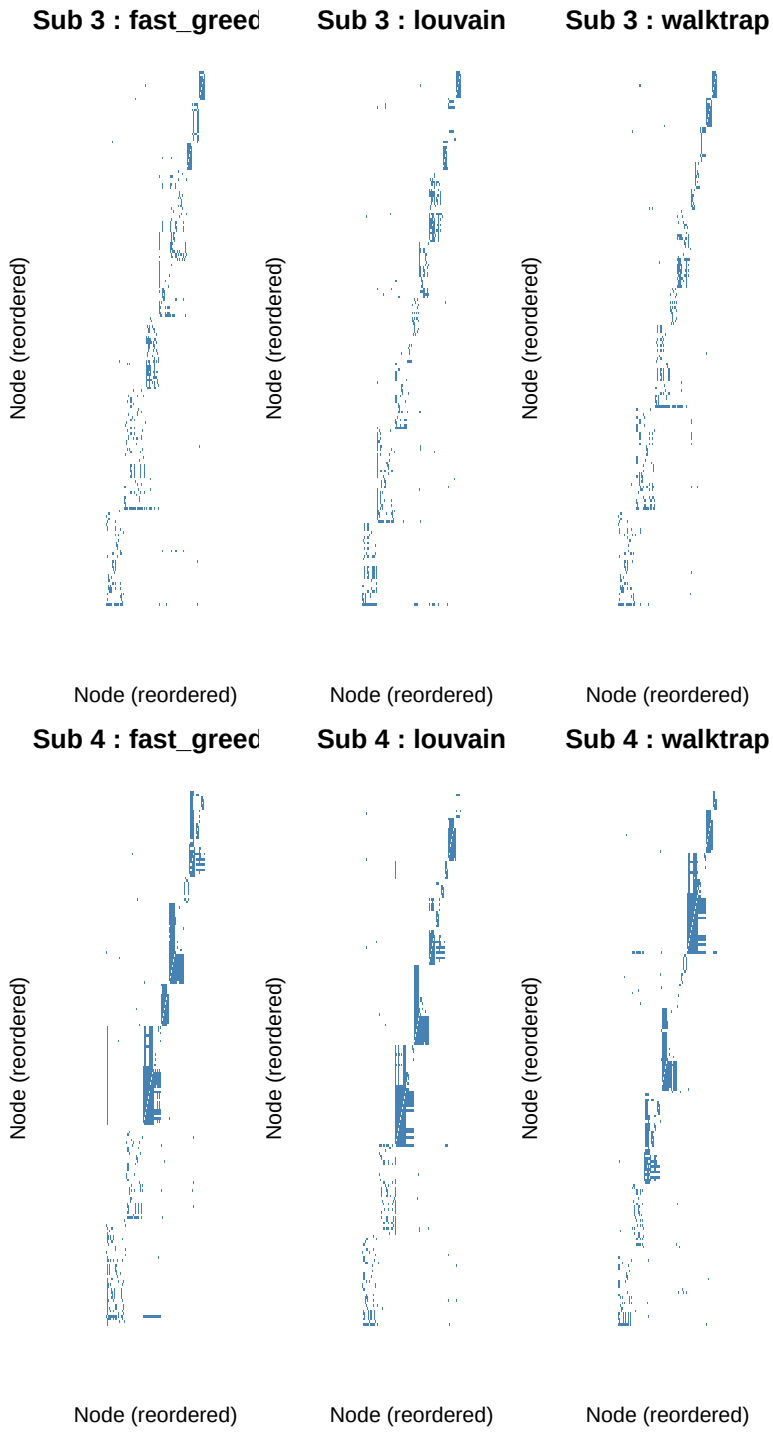

Subgraph 1 − Reordered by Community
Node (reordered)
Node (reordered)
Blue = hyperlink exists, White = no hyperlink


Subgraph 2 − Reordered by Community
Node (reordered)
Node (reordered)
Blue = hyperlink exists, White = no hyperlink


Subgraph 3 − Reordered by Community
Node (reordered)
Node (reordered)
Blue = hyperlink exists, White = no hyperlink


Subgraph 4 − Reordered by Community
Node (reordered)
Node (reordered)
Blue = hyperlink exists, White = no hyperlink


Sub 1 : fast_greed
Sub 1 : louvain
Sub 1 : walktrap
Sub 2 : fast_greed
Sub 2 : louvain
Sub 2 : walktrap
Node (reordered)
Node (reordered)


Sub 3 : fast_greed
Sub 3 : louvain
Sub 3 : walktrap
Sub 4 : fast_greed
Sub 4 : louvain
Sub 4 : walktrap
Node (reordered)
Node (reordered)